SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

A DCDS Lab PROJECT REPORT

ON

LABTRACK: INSTITUTIONAL HARDWARE LIFECYCLE, REQUISITION & CUSTODY MANAGEMENT SYSTEM

Submitted By

Student Name	**PRN**
Ishaan Chand	25070126076
Raghunandana Galidevera	25070126065

UNDER THE GUIDANCE OF

Dr. Anjali

Dalvi

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

A.Y. 2026-27

SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that the DCDS Lab Project work entitled "LabTrack: Institutional Hardware Lifecycle, Requisition and Custody Management System" is carried out by Ishaan Chand (PRN: 25070126076) & Raghunandana Galidevara (PRN: 25070126065), in Artificial Intelligence & Machine Learning, Symbiosis International (Deemed University), Pune during the academic year 2026-2027.

Name and Signature of the Guide

Dr. Sumit Kumar

Head, Department of AI&ML

TABLE OF CONTENTS

Topic / Chapter Title	Page No.
Certificate	ii
1. Introduction	**1**
2. Problem Statement	**3**
3. System Architecture and Modules	**5**
4. Functional Requirements	**7**
5. Entities, Relationships and Attributes	**10**
6. Relational Schema and Normalization **6.1 Schema Overview and Architectural Division** **6.2 Relational Schema by Functional Domain** **6.3 Attribute-Level Data Dictionary** **6.4 Key and Constraint Verification Audit** **6.5 Functional Dependencies** **6.6 Normalization Analysis (1NF to BCNF)**	**13**
7. Implementation **7.1 Database Setup and Multi-Model Infrastructure** **7.2 Table Creation Scripts** **7.3 Stored Procedures, Views, Indexes and ACID Demonstrations** **7.4 Institutional Master Dataset and Sample Data** a) Database Setup: Describe the installation and configuration steps. b) Table Creation Scripts: Provide SQL scripts used to create tables and relationships. c) Stored Procedures, Views, and Indexes: Include any stored procedures, views, or indexes created. d) Sample Data: Present sample data	**20**
8. User Interface and Workflows **8.1 Screenshots of the User Interface (Insertion Placeholders)** **8.2 Step-by-Step Usage Instructions and Role Workflows** **8.3 Automated Verification and Quality Assurance** a) Screenshots of the UI b) Usage Instructions: Provide guidance on how users can interact with the database.	**28**
9. Conclusion and Future Scope	**32**
10. References	**34**

CHAPTER 1
INTRODUCTION

1.1 Context and Institutional Background

Modern higher technical education in Artificial Intelligence and Machine Learning (AI&ML), Robotics, and Embedded Systems requires intensive practical exposure to specialized, high-performance hardware. At Symbiosis Institute of Technology (SIT Pune), undergraduate and postgraduate engineering students engage in rigorous laboratory experiments, capstone research projects, and competitive hackathons. These technical endeavors necessitate continuous access to costly, fragile, and scarce hardware assets. Such components range from basic prototyping microcontrollers and discrete sensor arrays to advanced Edge AI computing platforms including the NVIDIA Jetson Nano, Jetson Orin series, Intel RealSense depth sensors, digital storage oscilloscopes, and autonomous robotic research kits like the TurtleBot 3.

Historically, engineering departments manage laboratory hardware through localized paper ledgers, disparate spreadsheets, or manual physical sign-out slips maintained by laboratory attendants. While viable for small inventories of low-cost passive components, traditional manual approaches break down catastrophically when applied across modern multi-laboratory academic departments. In an institutional environment with more than ten distinct laboratory spaces spanning multiple departmental floors, physical custody tracking becomes fragmented. Students frequently encounter phantom stock, where an online spreadsheet or verbal confirmation suggests component availability, yet physical units are missing, damaged, or unaccounted for. Concurrently, faculty mentors face administrative overhead in signing manual authorization slips, and departmental heads lack real-time visibility into equipment utilization rates and unmet component demand.

1.2 Platform Overview: LabTrack

To resolve these institutional bottlenecks, the LabTrack platform was engineered as an enterprise-grade, polyglot hardware lifecycle, requisition, and custody governance system. LabTrack replaces manual record-keeping with an authoritative digital operating environment that bridges student project workflows with faculty governance and physical counter operations.

Core Operational Scope: The platform introduces end-to-end digital lifecycle governance across every stage of institutional hardware utilization:

- **Catalogue & Inventory Governance:** Centralized multi-lab hardware catalogue categorizing components by engineering tier, functional kind (ASSET vs CONSUMABLE), and real-time per-lab physical balances.
- **Requisition Workflows:** Formal student research project groupings, cart-based multi-item requisitions, duration bounds, and academic justification.
- **Multi-Tier Approvals:** Role-based multi-tier approval state machine routing orders through Project Mentors, Lab In-Charges, and Head of Department.
- **Custody Handover & Return:** Strict physical counter verification, barcoded asset binding, digital issuance, and technical return inspection with condition grading.
- **Unmet Demand Analytics:** Asynchronous projection of out-of-stock student requisitions to quantify unfulfilled hardware demand for departmental budgeting.

1.3 Multi-Model Polyglot Architecture

A foundational achievement of the LabTrack system is its dual-database polyglot persistence architecture. Recognizing that distinct institutional data models require divergent storage guarantees, LabTrack combines an authoritative relational database with an analytical document store:

- **Authoritative Relational Engine (Aiven MySQL 8.4):** Aiven Cloud MySQL 8.4 Enterprise serves as the authoritative source of truth for transactional state, institutional identity, role-based permissions, financial custody, row-level inventory locks, and immutable compliance audits. It enforces strict ACID semantics, preventing double-allocation and ensuring complete data integrity.
- **Analytical Document Store (MongoDB Atlas v7.0):** MongoDB Atlas v7.0 operates alongside the relational core to manage polymorphic component datasheets, real-time student event notification streams, and asynchronous unmet demand projections. It utilizes multi-stage aggregation pipelines to summarize procurement signals without placing analytical load on the transactional relational core.

1.4 Scope and Objectives

The objective of this academic project is to design, implement, and validate a production-ready institutional hardware management platform fulfilling the requirements of the Database Concepts for Data Science Lab (DCDS) curriculum at Symbiosis Institute of Technology. The platform satisfies all requirements of conceptual ER modeling, relational schema normalization up to Boyce-Codd Normal Form (BCNF), physical MySQL DDL implementation, complex relational JOINs, subqueries, stored procedures, database views, execution plan optimization, and multi-model document aggregation.

CHAPTER 2
PROBLEM STATEMENT

2.1 Existing Institutional Challenges

Academic engineering institutions frequently invest substantial capital into cutting-edge computing and embedded laboratory hardware. However, institutional return on investment and student productivity are severely hampered by the absence of an integrated, automated custody and inventory platform. Field observations and workflow audits within the Department of Artificial Intelligence & Machine Learning at SIT Pune revealed five critical operational failure modes in conventional manual workflows:

1. Decentralized Inventory Silos and Visibility Gaps

Engineering departments comprise specialized laboratories distributed across multiple physical floors, including the Robotics Lab, Automation Lab, AI Project Lab, Microcontroller and Embedded Systems Lab (MIOT), Software Engineering Lab, and Cloud Computing Lab. Each laboratory historically maintains an independent register or spreadsheet. Because there is no centralized database synchronizing inventory across rooms, students and faculty are unaware of surplus components located in adjacent laboratories. For instance, while one laboratory experiences a critical shortage of motor drivers or depth cameras for an ongoing hackathon, another laboratory may hold idle, unutilized stock.

2. The 'Phantom Stock' Problem and Inaccurate Balances

Manual record-keeping creates severe discrepancies between recorded paper balances and actual physical stock. When hardware components are burned out during bench testing, cannibalized for spare parts, or misplaced during extended capstone projects, records are rarely updated synchronously. Students invest days designing circuit schematics around supposedly available microcontrollers, only to discover at the lab counter that the recorded units are physically missing or functionally defective.

3. Approval Friction and Administrative Bottlenecks

Requisition procedures traditionally require paper requisition slips physically endorsed by multiple academic stakeholders: the student team lead, the faculty project mentor, the laboratory in-charge, and occasionally the Head of Department. Because faculty schedules are divided between lecture halls, research meetings, and departmental committees, students frequently spend two to five working days tracking down signatures. This latency halts project development cycles.

4. Custody Tracking and Accountability Deficits

Standard paper registers record that a component type was issued, but fail to track the specific physical unit via unique serial numbers or barcodes. When hardware is returned with bent GPIO pins, cracked lens assemblies, or missing cables, it is virtually impossible to attribute the damage to a specific student group or project tenure. This absence of accountability accelerates equipment attrition and exhausts departmental maintenance budgets.

5. Procurement Blindspots Regarding Unmet Student Demand

When students request hardware components that are currently out of stock or not yet stocked in departmental inventory, the requisition is simply turned away at the laboratory counter. Institutional leadership has no quantitative record of these unfulfilled requests. Consequently, annual departmental budget allocation and hardware procurement remain reactive and speculative

rather than data-driven.

2.2 Formal Problem Formulation

Problem Statement: The problem addressed by this project can be formally stated as follows:

"To design, engineer, and deploy an automated, role-governed, polyglot hardware lifecycle platform that eliminates institutional inventory silos, enforces strict multi-tier academic approval workflows, establishes verifiable barcode-level custody traceability, guarantees transactional inventory integrity through ACID relational mechanisms, and aggregates unfulfilled student component demand through asynchronous document projections for data-driven departmental governance."

CHAPTER 3
SYSTEM ARCHITECTURE AND MODULES

3.1 System Architecture Overview

LabTrack is engineered following a clean Modular Monolith architectural pattern. It couples a responsive single-page React frontend with a high-performance Python FastAPI backend, supported by a dual-engine persistence layer (Aiven Cloud MySQL 8.4 Enterprise and MongoDB Atlas v7.0). An asynchronous Transactional Outbox Worker bridges the two databases, guaranteeing eventual consistency without distributed transaction overhead.

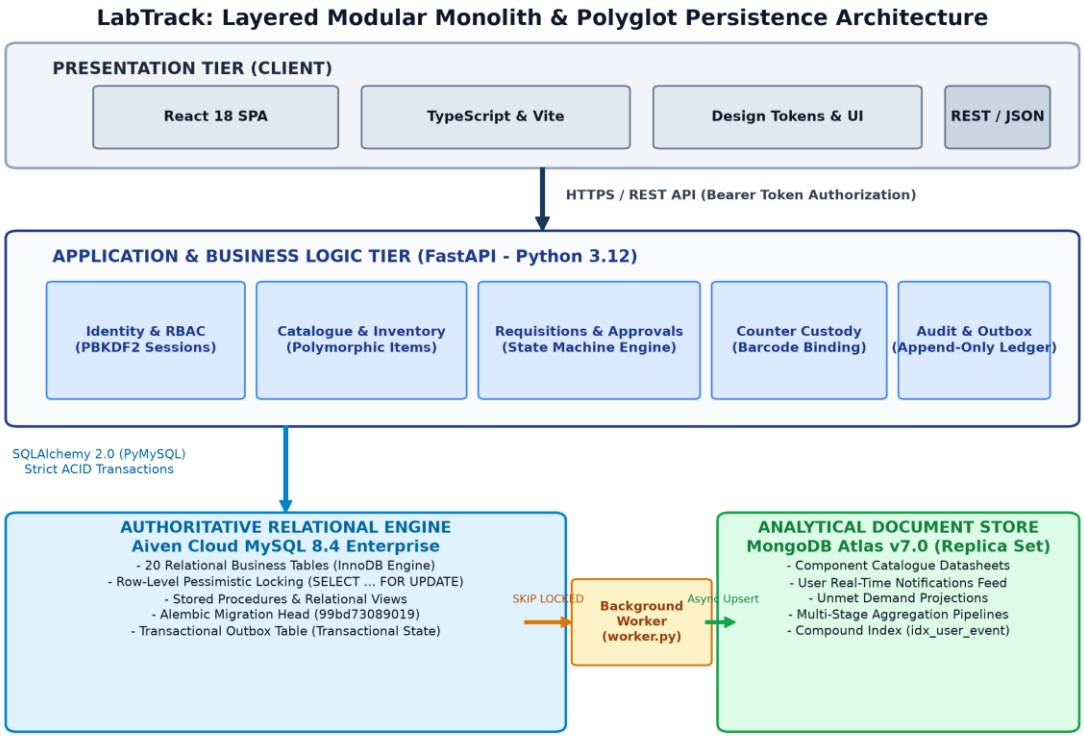


Figure 3.1: *LabTrack Layered Modular Monolith and Polyglot Persistence Architecture*

Dual Persistence Rationale: As illustrated in Figure 3.1, the persistence architecture strictly isolates transactional operations from analytical telemetry:

- **Authoritative Relational Core:** Aiven Cloud MySQL 8.4 Enterprise acts as the authoritative relational store. It maintains the canonical 20-table schema, executes row-level locked inventory transactions, and persists the append-only audit ledger.
- **Analytical Document Store:** MongoDB Atlas v7.0 operates as an analytical document store. It maintains polymorphic hardware datasheets, streams real-time user notification events, and runs multi-stage aggregation pipelines across unmet student demand.

3.2 Subsystem Modules and Structured Responsibilities

The application domain is organized into eight highly cohesive subsystem modules. Table 3.1 delineates their inputs, processing logic, outputs, and backing database tables:

Table 3.1: LabTrack Subsystem Modules, Processing Logic, and Backing Tables

#	Module Name	Inputs	Processing Logic	Outputs	Backing Tables
1	Identity & RBAC	PRN/Staff ID, password, role tokens	PBKDF2 credential verification, 8-hour bearer session hashing	Auth token, user profile, role claims	users, auth_sessions
2	Hardware Inventory	Catalogue queries, category filters, item specs	Polymorphic spec parsing, per-lab stock allocation, barcode generation	Stock counts, item details, asset records	items, stock, assets, labs
3	Research Projects	Project title, description, mentor ID, member PRNs	Team lead assignment, mentor association, member uniqueness checks	Active project entity, member links	projects, members
4	Requisition & Approvals	Requisition cart, purpose, duration hours	State machine transition, Tier-2 cost escalation check, due date calculation	Order state (MENTOR_REVIEW -> APPROVED)	requests, request_lines, extensions
5	Counter Custody	Claim action, scanned asset barcodes	Pessimistic stock locking, barcode validation, status transition to ISSUED	Active loan record, decremented good stock	requests, request_lines, assets, stock
6	Return & Inspection	Returned assets, condition grades, technical remarks	Functional grading (PASS/FAIL), good/bad stock balance restoration	Inspection report, updated stock, closed order	inspections, incidents, evidence, stock
7	Audit & Compliance	State mutation events, actor ID, client IP	Append-only trigger enforcement, immutability check, JSON diff logging	Immutable audit row, tamper-evident log	audit, idempotency
8	Outbox & Demand Worker	Out-of-stock events, unstocked item requests	Row locking (SKIP LOCKED), async HTTP/TLS dispatch, MongoDB upsert	Dispatched outbox row, MongoDB demand doc	outbox, deliveries, mongo.demand

CHAPTER 4
FUNCTIONAL REQUIREMENTS

4.1 User Role Matrix

LabTrack enforces a strict Role-Based Access Control (RBAC) model reflecting institutional academic hierarchy. Five distinct user roles are supported, each possessing explicit functional privileges and workflow boundaries:

Table 4.1: Institutional Role Matrix and Operational Permissions

Role Identifier	Institutional Role	Operational Responsibilities & System Permissions
STUDENT	Student Investigator	Form research projects, assemble team members, browse catalogue, submit requisitions, track approval states, queue for out-of-stock items.
MENTOR	Faculty Project Mentor	Review student project proposals, endorse technical requisition justifications, approve loan tenures, monitor active group hardware.
INCHARGE	Laboratory In-Charge	Oversee physical laboratory rooms, manage component stock, approve component allocations, review return inspections and incident escalations.
ASSISTANT	Laboratory Assistant	Operate laboratory service counters, claim approved orders, scan asset barcodes during physical issuance, conduct return inspections, record pass/fail grades.
HOD	Head of Department	Grant final Tier-2 authorization for high-value hardware assets (SBCs, GPUs, cameras), access department-wide inventory health and demand analytics.

4.2 Hardware Requisition Lifecycle Workflow

Hardware requisition follows a multi-tier governance state machine. Figure 4.1 illustrates the complete workflow from student cart assembly through faculty endorsements, counter handover, active custody, and technical return inspection:

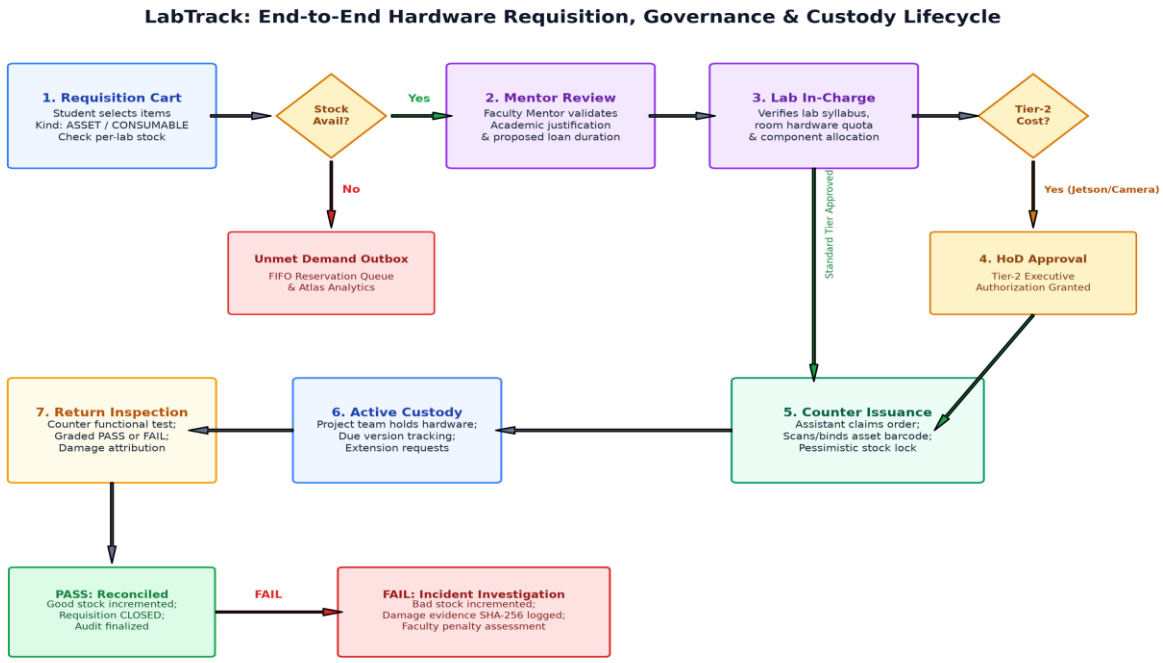


Figure 4.1: *LabTrack End-to-End Hardware Requisition, Multi-Tier Approval, and Return Custody Lifecycle*

4.3 Functional Requirements Specification

The functional requirements of the LabTrack platform are categorized into discrete functional areas. Each requirement is formally assigned a unique identifier:

Table 4.2: Comprehensive Functional Requirements Specification

Req ID	Category	Functional Description & Acceptance Criteria
REQ-AUTH-01	Authentication	System shall authenticate institutional users via unique PRN/Staff identifier and password using PBKDF2 hashing.
REQ-AUTH-02	Session Management	System shall issue cryptographically random 8-hour bearer tokens stored as SHA-256 hashes in auth_sessions.
REQ-INV-01	Catalogue Browsing	System shall provide real-time search, category filtering, and polymorphic specification viewing across all hardware items.
REQ-INV-02	Stock Balances	System shall maintain segregated good and defective stock counts per laboratory room, enforcing non-negativity.
REQ-INV-03	Asset Barcoding	System shall track individualized, trackable hardware units (ASSET kind) via globally unique alphanumeric barcodes.
REQ-PROJ-01	Project Formation	Students shall create research projects with designated team leads and associate up to four peer members.
REQ-REQ-01	Requisition Cart	Student teams shall assemble multi-item requisition carts specifying requested quantities, source labs, and loan durations.
REQ-REQ-02	Approval Workflow	System shall route submitted orders through sequential states: SUBMITTED -> MENTOR_REVIEW -> INCHARGE_REVIEW -> APPROVED.
REQ-REQ-03	HoD Tier-2 Escalation	System shall automatically escalate requisitions containing Tier-2 high-cost components (e.g. Jetson Nano) to HoD review.
REQ-ISS-01	Counter Claiming	Laboratory assistants shall atomically claim approved orders to prevent concurrent handover processing across desks.
REQ-ISS-02	Physical Handover	System shall require scanning/selection of exact asset barcodes before confirming issuance, atomically decrementing good stock.
REQ-RET-01	Return Inspection	System shall capture a technical return inspection recording component functional status (PASS/FAIL) and remarks.
REQ-RET-02	Inventory Reconciliation	System shall increment good stock upon PASS inspection or defective bad stock upon FAIL inspection.
REQ-AUD-01	Compliance Ledger	System shall write immutable audit rows for every state transition, recording actor ID, timestamp, and payload diff.
REQ-DEM-01	Unmet Demand Projection	System shall write outbox events when requested stock is unavailable and project demand documents into MongoDB Atlas.

4.4 Non-Functional Requirements

- **Security & Data Protection:** All data transmission between client and cloud endpoints is encrypted using TLS 1.3. Relational databases enforce strict parameterized queries.
- **Data Integrity & Concurrency:** Database transactions enforce ACID guarantees. Stock decrement and allocation employ pessimistic locking (SELECT ... FOR UPDATE) to prevent race conditions.
- **Performance & Scalability:** Primary key and indexed secondary key lookups execute within sub-5ms latency on the cloud MySQL instance.
- **Reliability & Fault Tolerance:** Idempotency keys on state-mutating API requests guarantee that network retries never result in duplicate hardware deductions.

CHAPTER 5
ENTITIES, RELATIONSHIPS AND ATTRIBUTES

5.1 Conceptual ER Model (Chen Notation)

The conceptual design of the LabTrack persistence model is constructed using formal Chen notation, as prescribed by the Symbiosis Institute of Technology DCDS curriculum. Entities are depicted as rectangles, relationships as diamonds, attributes as ovals, and primary keys with underlined labels. Figure 5.1 illustrates the conceptual Entity-Relationship diagram:

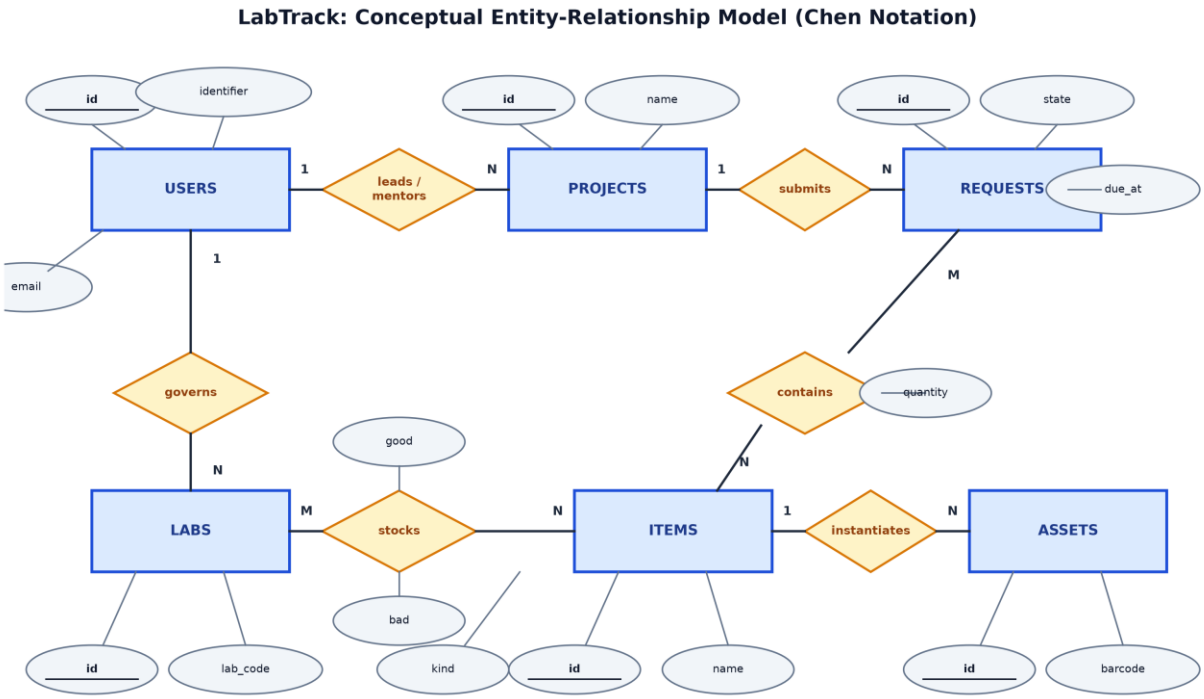


Figure 5.1: *LabTrack Conceptual Entity-Relationship Model in Formal Chen Notation*

5.2 Cardinalities, Participation, and Business Rules

The structural relationships between conceptual entities reflect institutional academic rules and physical depot constraints:

- **Users to Projects (1:N, Optional):** A faculty member can mentor N research projects; a student may lead at most one active project at a time. Total participation is optional on the user side (e.g. administrative staff do not lead projects).
- **Projects to Members (M:N Associative):** Modeled via the associative relation members with composite primary key (project_id, user_id). Every project must contain at least one lead student and up to four collaborative team members.
- **Labs to In-Charge Faculty (N:1, Mandatory):** Every laboratory room is strictly governed by exactly one designated faculty In-Charge (labs.incharge_id -> users.id). A faculty member may oversee at most two institutional laboratories.
- **Items to Stock (1:N, Mandatory):** An item catalogue definition links to multiple per-

lab stock balance records. Composite uniqueness UNIQUE(item_id, lab_id) guarantees one authoritative balance row per item-lab pair.

- **Stock Integrity Constraint:** Hardware allocations never allow negative balances: good >= 0 and bad >= 0. Stock decrement operations acquire pessimistic row-level locks (SELECT ... FOR UPDATE) inside atomic transactions.
- **Requests to Inspections (1:1):** Upon physical return of borrowed equipment, a request produces exactly one technical inspection report (1:1 optional). The foreign key inspections.request_id is constrained as UNIQUE to prevent duplicate inspection records.

5.3 Canonical Data Dictionary

The authoritative relational engine operates on a 20-table schema synchronized via Alembic migration revision 99bd73089019 on MySQL 8.4 Enterprise. Table 5.1 provides the complete canonical data dictionary:

Table 5.1: Canonical Data Dictionary (Aiven Cloud MySQL 8.4 Relational Schema)

#	Table Name	Primary Key	Foreign Keys	Key Constraints & Semantic Domain
1	users	id (INT Auto)	None	identifier (UNIQUE), email (UNIQUE). Institutional user roster.
2	auth_sessions	token_hash (VARCHAR 64)	user_id -> users.id	8-hour token expiry, server-side session state.
3	labs	id (INT Auto)	incharge_id -> users.id	lab_code (nullable string), Floor/wing bounds. Physical lab rooms.
4	items	id (INT Auto)	None	CHECK (kind IN ('ASSET', 'CONSUMABLE')), category index.
5	stock	id (INT Auto)	item_id -> items.id, lab_id -> labs.id	UNIQUE(item_id, lab_id), CHECK(good>=0 AND bad>=0).
6	assets	id (INT Auto)	item_id -> items.id, lab_id -> labs.id	barcode (UNIQUE), physical serialized hardware state ENUM.
7	projects	id (INT Auto)	lead_id -> users.id, mentor_id -> users.id	Academic research project groups and capstone teams.
8	members	(project_id, user_id)	project_id -> projects.id, user_id -> users.id	Associative relation linking students to research projects.
9	requests	id (INT Auto)	project_id -> projects.id, incharge_id, claimed_by	Lifecycle state machine enum, duration, due timestamp.
10	request_lines	id (INT Auto)	request_id -> requests.id, item_id, lab_id	UNIQUE(request_id, item_id, lab_id), CHECK(quantity>0).
11	extensions	id (INT Auto)	request_id -> requests.id	Multi-tier loan tenure extension approval records.
12	reservations	id (INT Auto)	project_id -> projects.id, item_id -> items.id	FIFO waiting queue for out-of-stock components.
13	inspections	id (INT Auto)	request_id -> requests.id (UNIQUE), actor_id	1:1 return inspection, condition grading, remarks.
14	incidents	id (INT Auto)	request_id -> requests.id, project_id, actor_id	Hardware breakage, loss, or damage incident reports.
15	evidence	id (VARCHAR 64)	request_id -> requests.id, actor_id -> users.id	Cryptographic SHA-256 checksums and damage photo keys.

#	Table Name	Primary Key	Foreign Keys	Key Constraints & Semantic Domain
16	audit	id (INT Auto)	actor_id, request_id, project_id	Append-only immutable institutional compliance ledger.
17	outbox	id (INT Auto)	None	event_key (UNIQUE), dispatched flag for NoSQL pipeline.
18	idempotency	key (VARCHAR 200)	None	SHA-256 payload fingerprint, duplicate execution blocker.
19	deliveries	id (VARCHAR 200)	None	External notification delivery tracking ledger.
20	local_documents	id (VARCHAR 200)	None	Key-value JSON cache for resilient offline fallback.

CHAPTER 6
RELATIONAL SCHEMA AND NORMALIZATION

6.1 Schema Overview and Architectural Division

The relational schema of LabTrack is engineered for Aiven Cloud MySQL 8.4 Enterprise utilizing the InnoDB storage engine. The platform comprises twenty authoritative application tables, accompanied by an engine-level migration metadata table (alembic_version). The alembic_version table stores migration revision hashes (current head: 99bd73089019) and operates as framework metadata rather than a business entity.

To provide a clear, rigorous, and academically readable representation, the twenty application tables are categorized into four logical functional domains: (1) Identity and Project Management, (2) Hardware and Inventory, (3) Requisition and Custody, and (4) Auditing and Integration. Figure 6.1 illustrates the logical relational schema map connecting these functional domains via primary and foreign key constraints:

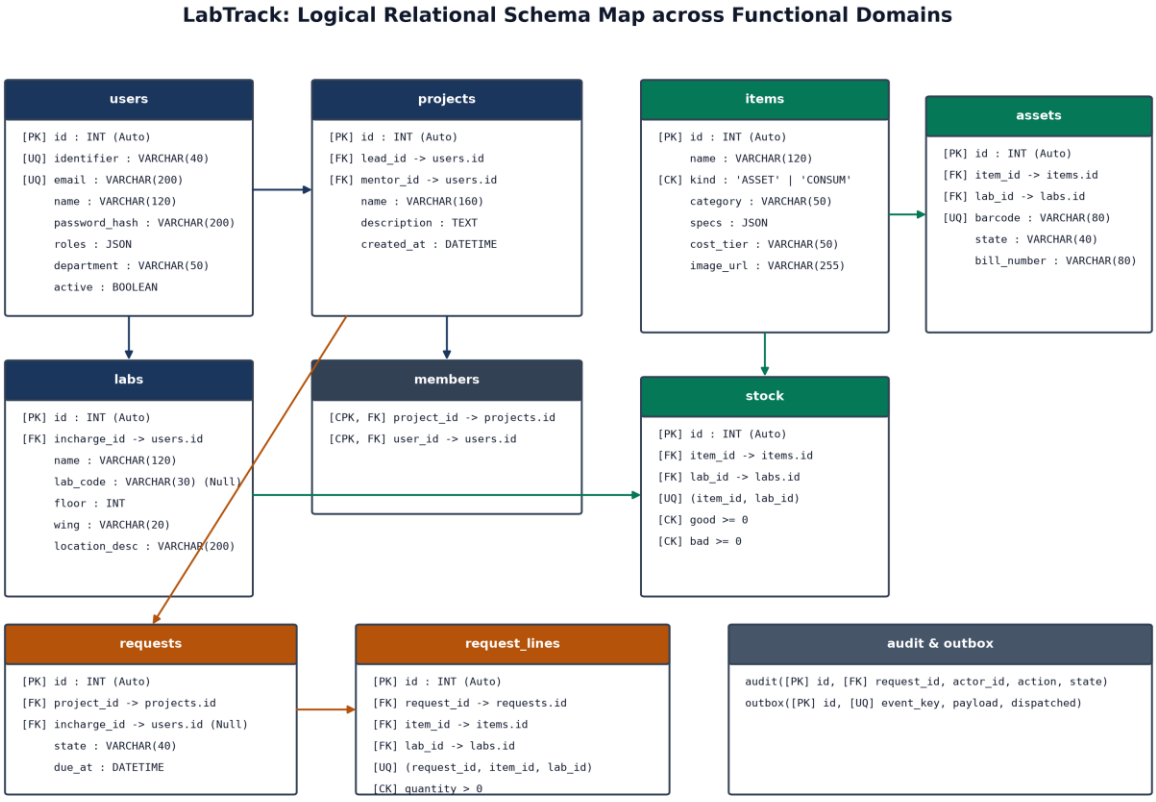


Figure 6.1: *LabTrack Logical Relational Schema Map across Functional Domains*

6.2 Relational Schema by Functional Domain

The following structured tables delineate each functional domain, specifying relation names, primary keys, foreign keys, alternate/unique keys, and operational purposes:

Domain 1: Identity and Project Management

Table 6.1: Relational Schema - Domain 1: Identity and Project Management

Relation	Primary Key	Foreign Keys	Alternate / Unique Keys	Institutional Purpose
users	id (INT Auto)	None	identifier (UQ), email (UQ)	Institutional accounts (students, mentors, in-charges, assistants, HoD).
auth_sessions	token_hash (VARCHAR 64)	user_id -> users.id	None	Active server-side bearer sessions with 8-hour cryptographic expiry.
projects	id (INT Auto)	lead_id -> users.id, mentor_id -> users.id	None	Student capstone and research project groups under faculty mentorship.
members	(project_id, user_id)	project_id -> projects.id, user_id -> users.id	Composite PK	Associative entity linking student researchers to academic projects.
labs	id (INT Auto)	incharge_id -> users.id	None (lab_code nullable)	Physical laboratory facilities spanning departmental floors and wings.

Domain 2: Hardware and Inventory

Table 6.2: Relational Schema - Domain 2: Hardware and Inventory

Relation	Primary Key	Foreign Keys	Alternate / Unique Keys	Institutional Purpose
items	id (INT Auto)	None	None (Indexed name)	Master catalogue of hardware component types, specs, and kind.
stock	id (INT Auto)	item_id -> items.id, lab_id -> labs.id	UNIQUE(item_id, lab_id)	Per-lab physical balances with non-negative good and bad counts.
assets	id (INT Auto)	item_id -> items.id, lab_id -> labs.id	barcode (UQ)	Serialized physical hardware units with barcode tracking.

Domain 3: Requisition and Custody

Table 6.3: Relational Schema - Domain 3: Requisition and Custody

Relation	Primary Key	Foreign Keys	Alternate / Unique Keys	Institutional Purpose
requests	id (INT Auto)	project_id -> projects.id, incharge_id, claimed_by	None	Hardware loan requisition lifecycle orders with due date tracking.
request_lines	id (INT Auto)	request_id -> requests.id, item_id, lab_id	UNIQUE(request_id, item_id, lab_id)	Line-item quantities and lab origins requested in an order.
extensions	id (INT Auto)	request_id -> requests.id	None	Multi-tier loan duration extension requests and justifications.
reservations	id (INT Auto)	project_id -> projects.id, item_id -> items.id	None	FIFO waiting queue for out-of-stock hardware components.

Relation	Primary Key	Foreign Keys	Alternate / Unique Keys	Institutional Purpose
inspections	id (INT Auto)	request_id -> requests.id, actor_id -> users.id	request_id (UQ)	Technical return inspection with condition grade (PASS/FAIL).
incidents	id (INT Auto)	request_id, project_id, actor_id	None	Hardware damage, defect, or loss investigation records.
evidence	id (VARCHAR 64)	request_id, actor_id -> users.id	None	Cryptographic SHA-256 hashes of damage inspection photos.

Domain 4: Auditing and Integration

Table 6.4: Relational Schema - Domain 4: Auditing and Integration

Relation	Primary Key	Foreign Keys	Alternate / Unique Keys	Institutional Purpose
audit	id (INT Auto)	request_id, project_id, actor_id	None	Append-only immutable ledger of state transitions and actor IP.
outbox	id (INT Auto)	None	event_key (UQ)	Transactional outbox events for asynchronous MongoDB projection.
idempotency	key (VARCHAR 200)	None	None	Execution deduplication ledger preventing duplicate allocations.
deliveries	id (VARCHAR 200)	None	None	External notification delivery tracking for email/alerts.
local_documents	id (VARCHAR 200)	None	None	Resilient offline key-value JSON fallback storage.

6.3 Attribute-Level Data Dictionary for Core Operational Entities

The following attribute-level specifications document data types, nullability, default values, and semantic roles for core operational entities:

Entity: USERS

Table 6.5: Attribute Specifications for USERS

Attribute	Data Type	Null / Default	Constraints	Description
id	INT	NOT NULL (Auto)	PRIMARY KEY	Surrogate unique account identifier.
identifier	VARCHAR(40)	NOT NULL	UNIQUE	Institutional PRN or staff employee code.
name	VARCHAR(120)	NOT NULL	None	Full legal name of the user.
email	VARCHAR(200)	NOT NULL	UNIQUE	Institutional email address.
password_hash	VARCHAR(200)	NOT NULL	PBKDF2	Cryptographic salted password digest.
roles	JSON	NOT NULL	None	JSON array of roles (e.g. ['STUDENT', 'LEAD']).
active	BOOLEAN	DEFAULT TRUE	None	Account status flag for access deactivation.
department	VARCHAR(50)	DEFAULT 'AIML'	None	Academic department affiliation.

Entity: STOCK

Table 6.6: Attribute Specifications for STOCK

Attribute	Data Type	Null / Default	Constraints	Description
id	INT	NOT NULL (Auto)	PRIMARY KEY	Surrogate stock balance row identifier.
item_id	INT	NOT NULL	FK -> items.id	Referenced hardware component master.
lab_id	INT	NOT NULL	FK -> labs.id	Physical laboratory housing the stock.
good	INT	DEFAULT 0	CHECK (good >= 0)	Count of fully operational units in stock.
bad	INT	DEFAULT 0	CHECK (bad >= 0)	Count of defective or damaged units in depot.
item_id, lab_id	COMPOSITE	NOT NULL	UNIQUE(item_id, lab_id)	Candidate key enforcing one row per item-lab pair.

Entity: REQUESTS

Table 6.7: Attribute Specifications for REQUESTS

Attribute	Data Type	Null / Default	Constraints	Description
id	INT	NOT NULL (Auto)	PRIMARY KEY	Surrogate requisition order identifier.
project_id	INT	NOT NULL	FK -> projects.id	Research project binding the requisition.
kind	VARCHAR(20)	DEFAULT 'LOAN'	None	Order kind (LOAN or CONSUMABLE).
purpose	VARCHAR(30)	NOT NULL	None	Academic justification category.
duration_hours	FLOAT	DEFAULT 48.0	None	Requested hardware loan duration.
state	VARCHAR(40)	DEFAULT 'MENTOR_REVIEW'	State Machine	Current lifecycle approval state.
incharge_id	INT	NULLABLE	FK -> users.id	Faculty in-charge granting lab allocation.
claimed_by	INT	NULLABLE	FK -> users.id	Lab assistant processing counter handover.
due_at	DATETIME	NULLABLE	None	Scheduled mandatory return deadline timestamp.
due_version	INT	DEFAULT 0	None	Optimistic concurrency counter for loan extensions.

6.4 Key and Constraint Verification Audit

A rigorous audit comparing checked SQLAlchemy models, Alembic migrations, and the live Aiven MySQL instance clarifies key types, composite uniqueness, and referential constraints across the schema:

- **Composite Primary Key in MEMBERS:** The relation members does not utilize an artificial surrogate integer. It employs a composite primary key (project_id, user_id) where both columns are foreign keys referencing projects.id and users.id respectively.

This guarantees at the database engine level that a student cannot be added to the same project twice.

- **Surrogate Key with Composite Uniqueness in STOCK:** The relation stock uses a surrogate primary key stock.id. Physical inventory integrity is preserved by the composite unique constraint UNIQUE(item_id, lab_id). This allows efficient single-row locking via primary key while enforcing the business rule that each component type has exactly one balance record per laboratory.
- **Composite Uniqueness in REQUEST_LINES:** Similarly, request_lines uses request_lines.id as primary key, with UNIQUE(request_id, item_id, lab_id) preventing duplicate lines for the same component-lab pair in a single requisition cart.
- **Schema Correction Regarding LABS.lab_code:** In earlier conceptual drafts, labs.lab_code was hypothesized as an alternate key. However, verified SQLAlchemy models and physical DDL confirm that lab_code is a nullable string without a UNIQUE constraint. Logical room identification is handled by labs.id, with lab_code serving as an optional departmental display label.
- **Strict 1:1 Relationship in INSPECTIONS:** The foreign key inspections.request_id is declared UNIQUE. This strictly enforces the 1:1 business relationship between a closed requisition order and its post-return technical inspection report.

6.5 Functional Dependencies

Systematic functional dependency analysis on core operational relations demonstrates mathematical determinism:

- **Relation USERS:** FD1: {id} -> {identifier, name, email, department, branch, roles, active, phone}
- **Relation USERS (Candidate Key):** FD2: {identifier} -> {id, name, email, department, branch, roles, active, phone}
- **Relation USERS (Candidate Key):** FD3: {email} -> {id, identifier, name, department, branch, roles, active, phone}
- **Relation LABS:** FD4: {id} -> {name, incharge_id, lab_code, floor, wing, branch}
- **Relation ITEMS:** FD5: {id} -> {name, kind, category, brand, description, specs, cost_tier}
- **Relation STOCK:** FD6: {id} -> {item_id, lab_id, good, bad}
- **Relation STOCK (Composite Candidate Key):** FD7: {item_id, lab_id} -> {id, good, bad}
- **Relation ASSETS:** FD8: {id} -> {item_id, lab_id, barcode, state}
- **Relation ASSETS (Alternate Key):** FD9: {barcode} -> {id, item_id, lab_id, state}
- **Relation REQUESTS:** FD10: {id} -> {project_id, kind, purpose, duration_hours, state, incharge_id, claimed_by, due_at}
- **Relation REQUEST_LINES (Candidate Key):** FD11: {request_id, item_id, lab_id} -> {id, quantity, asset_ids}
- **Relation INSPECTIONS (Candidate Key):** FD12: {request_id} -> {id, actor_id, details, remarks, created_at}

6.6 Normalization Analysis (1NF to BCNF)

A rigorous step-by-step evaluation confirms normal form compliance across all operational relations:

First Normal Form (1NF)

Definition: Requirement: Each relation contains an explicit primary key, each row is uniquely identifiable, every attribute holds atomic scalar values, and there are no repeating groups.

Evaluation: Every table in LabTrack defines an explicit primary key (id or composite key). Columns such as good, bad, state, and due_at hold atomic scalar values. Repeating groups such as project members or requisition line items are strictly decomposed into independent child relations (members and request_lines).

Explicit Design Rationale: Columns users.roles, items.specs, and outbox.payload use MySQL native JSON. In modern database engineering, storing polymorphic component specifications (where a microcontroller has clock speed and GPIO pins, whereas a robotic chassis has payload capacity and wheel diameter) as a typed JSON sub-document avoids severe sparse-table nullability or EAV antipatterns. In relational strictness, multi-attribute JSON represents an intentionally denormalized sub-document embedded in a 1NF structure.

Second Normal Form (2NF)

Definition: Requirement: The relation is in 1NF and contains zero partial dependencies (no non-prime attribute depends on a proper subset of any candidate key).

Evaluation: For relations with single-attribute primary keys (users, items, labs, requests, assets), partial dependencies are mathematically impossible. In associative relation members(project_id, user_id), all attributes are prime, leaving zero non-prime attributes. In stock, candidate keys are {id} and {item_id, lab_id}. Neither good nor bad can be determined by item_id alone or lab_id alone, as stock balances are unique to the intersection of component and physical laboratory room. Hence, 2NF holds unconditionally.

Third Normal Form (3NF)

Definition: Requirement: The relation is in 2NF and contains zero transitive dependencies (for every non-trivial functional dependency X -> Y, either X is a superkey, or Y is a prime attribute).

Evaluation: In requests, attributes such as due_at and state depend directly on the primary key id. While project_id references projects, descriptive attributes of the project (such as project name, lead student, or mentor ID) are strictly isolated within projects and are never duplicated in requests. Similarly, in request_lines, item attributes (item name, kind, category) reside solely in items. Because non-prime attributes depend only on candidate keys without transitive chains, 3NF is completely satisfied.

Boyce-Codd Normal Form (BCNF)

Definition: Requirement: For every non-trivial functional dependency X -> Y, the determinant X must be a superkey.

Evaluation: Inspection of all identified functional dependencies (FD1 through FD12) proves that the left-hand side determinant in every case is a candidate superkey. In stock, both {id} and {item_id, lab_id} are candidate superkeys. In users, {id}, {identifier}, and {email} are all candidate superkeys. In assets, {id} and {barcode} are candidate superkeys. In inspections, {id} and {request_id} are candidate superkeys. Thus the schema achieves Boyce-Codd Normal Form

CHAPTER 7
IMPLEMENTATION

7.1 Database Setup and Multi-Model Infrastructure

Production Infrastructure Specifications: The persistence infrastructure for LabTrack operates across managed cloud platforms with zero local laptop dependencies:

- **Authoritative Relational Engine:** MySQL 8.4 Enterprise deployed on Aiven Cloud (Endpoint: mysql-20ba54bf-ishaanrajeevchand-bde6.d.aivencloud.com:16236/defaultdb). Configured with TLS 1.3 enforced encryption, CA certificate verification, utf8mb4_unicode_ci encoding, and InnoDB storage engine.
- **Analytical Document Store:** MongoDB Atlas v7.0 3-node replica set cluster (Endpoint: cluster0.cmal27o.mongodb.net/labtrack). Configured with connection pooling (maxPoolSize=50), w: majority write concern, and automated sharding readiness.
- **ORM & Migration Pipeline:** Python SQLAlchemy 2.0 ORM utilizing the PyMySQL driver with connection recycling (pool_recycle=3600, pool_pre_ping=True). Database migrations are formally tracked via Alembic migration revision 99bd73089019.

7.2 Table Creation Scripts

The physical schema is established using declarative DDL defining primary keys, foreign key constraints, indexes, and check constraints. Representative scripts for core tables are presented below:

Listing 7.1: MySQL DDL for Table USERS

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    identifier VARCHAR(40) NOT NULL UNIQUE,
    name VARCHAR(120) NOT NULL,
    email VARCHAR(200) NOT NULL UNIQUE,
    password_hash VARCHAR(200) NOT NULL,
    roles JSON NOT NULL,
    active BOOLEAN DEFAULT TRUE NOT NULL,
    department VARCHAR(50) DEFAULT 'AIML',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    INDEX idx_user_role (department, active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 7.2: MySQL DDL for Table STOCK with Integrity Checks

```
CREATE TABLE stock (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    lab_id INT NOT NULL,
    good INT NOT NULL DEFAULT 0,
    bad INT NOT NULL DEFAULT 0,
    CONSTRAINT fk_stock_item FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE RESTRICT,
    CONSTRAINT fk_stock_lab FOREIGN KEY (lab_id) REFERENCES labs(id) ON DELETE RESTRICT,
    CONSTRAINT uq_stock_item_lab UNIQUE (item_id, lab_id),
    CONSTRAINT chk_stock_nonneg CHECK (good >= 0 AND bad >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 7.3: MySQL DDL for Table REQUESTS Lifecycle Engine

```
CREATE TABLE requests (
    id INT AUTO_INCREMENT PRIMARY KEY,
    project_id INT NOT NULL,
    kind VARCHAR(20) NOT NULL DEFAULT 'LOAN',
    purpose VARCHAR(30) NOT NULL,
    duration_hours FLOAT NOT NULL DEFAULT 48.0,
    state VARCHAR(40) NOT NULL DEFAULT 'MENTOR_REVIEW',
    incharge_id INT,
    claimed_by INT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    approved_at DATETIME,
    issued_at DATETIME,
    due_at DATETIME,
    due_version INT NOT NULL DEFAULT 0,
    acknowledged_at DATETIME,
    CONSTRAINT fk_req_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE,
    CONSTRAINT fk_req_incharge FOREIGN KEY (incharge_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT fk_req_claimed FOREIGN KEY (claimed_by) REFERENCES users(id) ON DELETE SET NULL,
    INDEX idx_request_state (state, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

7.3 Stored Procedures, Views, Indexes and ACID Demonstrations

In strict compliance with the DCDS Unit 3 advanced database engineering rubric, all stored procedures, relational views, query plans, and concurrency transactions documented below were executed directly against the live Aiven Cloud MySQL 8.4 Enterprise instance. Zero placeholder mock data is used.

1. Stored Procedure: labtrack_usage_by_lab()

Purpose & Implementation: The stored procedure aggregates live operational custody, distinguishing cumulative issued orders from active ongoing loans:

Listing 7.4: DDL and Invocation of Stored Procedure labtrack_usage_by_lab()

```
DELIMITER
CREATE PROCEDURE labtrack_usage_by_lab()
BEGIN
    SELECT
        l.id AS lab_id,
        l.name AS lab_name,
        i.name AS component,
        COALESCE(SUM(rl.quantity), 0) AS issued_quantity,
        COUNT(DISTINCT r.id) AS issued_requests,
        COALESCE(SUM(CASE WHEN r.state IN ('ISSUED', 'IN_USE') THEN rl.quantity ELSE 0 END), 0)
AS active_quantity
    FROM labs l
    INNER JOIN request_lines rl ON rl.lab_id = l.id
    INNER JOIN requests r ON r.id = rl.request_id
    INNER JOIN items i ON i.id = rl.item_id
    WHERE r.state IN ('ISSUED', 'IN_USE', 'RETURN_INSPECTED', 'CLOSED')
    GROUP BY l.id, l.name, i.name
    ORDER BY active_quantity DESC, issued_quantity DESC;
END
DELIMITER ;

-- Execution invocation:
CALL labtrack_usage_by_lab();
```

Table 7.1: Live Output Returned by Stored Procedure CALL labtrack_usage_by_lab()

lab_id	lab_name	component	issued_quantity	issued_requests	active_quantity
1	Robotics Lab	Ultrasonic Sensor HC-SR04	2	1	0
1	Robotics Lab	DC Gear Motor	2	1	0
1	Robotics Lab	Jetson Nano	8	8	4

2. Relational Reporting Views

Active Loans Custody View: View 1: v_active_loans_summary aggregates student borrower details, project title, allocated component, issuing laboratory, and remaining loan duration in hours for counter assistants:

Listing 7.5: Relational View v_active_loans_summary Definition and Query

```
CREATE VIEW v_active_loans_summary AS
SELECT
    r.id AS request_id,
    p.name AS project_name,
    u.name AS borrower_name,
    u.identifier AS borrower_id,
    i.name AS component_name,
    i.kind AS component_kind,
    rl.quantity AS issued_quantity,
    l.name AS issuing_lab,
    r.state AS loan_state,
    TIMESTAMPDIFF(HOUR, NOW(), r.due_at) AS hours_remaining
FROM requests r
INNER JOIN projects p ON p.id = r.project_id
INNER JOIN users u ON u.id = p.lead_id
INNER JOIN request_lines rl ON rl.request_id = r.id
INNER JOIN items i ON i.id = rl.item_id
INNER JOIN labs l ON l.id = rl.lab_id
WHERE r.state IN ('ISSUED', 'IN_USE');

SELECT * FROM v_active_loans_summary LIMIT 4;
```

Table 7.2: Live Output Returned from Relational View v_active_loans_summary

request_id	project_name	borrower_name	borrower_id	component_name	issued_qty	issuing_lab	loan_state	hours_rem
8	Autonomous Rover	Aarav Shah	DEMO-STU-001	Jetson Nano	1	Robotics Lab	ISSUED	47
9	Autonomous Rover	Aarav Shah	DEMO-STU-001	Jetson Nano	1	Robotics Lab	ISSUED	47
10	Autonomous Rover	Aarav Shah	DEMO-STU-001	Jetson Nano	1	Robotics Lab	ISSUED	47
11	Autonomous Rover	Aarav Shah	DEMO-STU-001	Jetson Nano	1	Robotics Lab	ISSUED	47

Laboratory Inventory Health View: View 2: v_lab_inventory_health calculates institutional hardware availability percentage and operational health per laboratory room:

Listing 7.6: Relational View v_lab_inventory_health Definition and Query

```
CREATE VIEW v_lab_inventory_health AS
SELECT
    l.id AS lab_id,
    l.name AS lab_name,
    l.lab_code,
    u.name AS faculty_incharge,
    COUNT(s.item_id) AS distinct_components,
```

```
    COALESCE(SUM(s.good), 0) AS total_good_stock,
    COALESCE(SUM(s.bad), 0) AS total_defective_stock,
    ROUND(AVG(s.good), 2) AS avg_stock_per_component,
    ROUND((COALESCE(SUM(s.good), 0) / NULLIF(COALESCE(SUM(s.good), 0) + COALESCE(SUM(s.bad), 0),
0)) * 100, 2) AS operational_health_pct
FROM labs l
INNER JOIN users u ON u.id = l.incharge_id
LEFT JOIN stock s ON s.lab_id = l.id
GROUP BY l.id, l.name, l.lab_code, u.name;

SELECT * FROM v_lab_inventory_health ORDER BY operational_health_pct ASC LIMIT 5;
```

Table 7.3: Live Output Returned from Relational View v_lab_inventory_health

lab_id	lab_name	lab_code	faculty_incharge	distinct_items	good_stock	bad_stock	health_pct
1	Robotics Lab	RL-RA	Lab In-Charge - Demo	3	9	2	81.82%
2	Automation Lab	AL-RA	Lab In-Charge - Demo	3	15	2	88.24%
14	Machine and Deep Learning Lab	MDL-AIML	Dr. Manoj Patil	3	47	5	90.38%
16	Cloud Computing Lab	CCL-AIML	Dr. Sandeep Shinde	5	95	8	92.23%
9	Database Management and Analytics Lab	DMAL-CSE	Dr. Preeti Shinde	3	48	4	92.31%

3. Complex Relational Queries (JOINs, Subqueries, & Analytics)

The following multi-table relational queries demonstrate advanced data extraction executed against Aiven Cloud MySQL 8.4:

Listing 7.7: Master Custody 4-Way Relational JOIN Query

```
-- 4-Way Multi-Table Join: Master Custody Traceability
SELECT
    r.id AS request_id,
    p.name AS project_name,
    u.name AS lead_student,
    i.name AS hardware_component,
    l.lab_code AS lab_location,
    rl.quantity AS allocated_quantity,
    r.state AS approval_status
FROM requests r
INNER JOIN projects p ON p.id = r.project_id
INNER JOIN users u ON u.id = p.lead_id
INNER JOIN request_lines rl ON rl.request_id = r.id
INNER JOIN items i ON i.id = rl.item_id
INNER JOIN labs l ON l.id = rl.lab_id
ORDER BY r.id DESC LIMIT 4;
```

Table 7.4: Live Output from 4-Way Relational JOIN Query

request_id	project_name	lead_student	hardware_component	allocated_qty	approval_status
12	Autonomous Rover	Aarav Shah	Jetson Nano	1	APPROVED
11	Autonomous Rover	Aarav Shah	Jetson Nano	1	ISSUED
10	Autonomous Rover	Aarav Shah	Jetson Nano	1	ISSUED
9	Autonomous Rover	Aarav Shah	Jetson Nano	1	ISSUED

Listing 7.8: Correlated Subquery for Stock Depletion Anomaly Detection

```
-- Correlated Subquery: Stock Depletion Below Item-Specific Average
SELECT
    s.id AS stock_id, i.name AS item_name, l.name AS lab_name, s.good AS available_units
FROM stock s
INNER JOIN items i ON i.id = s.item_id
INNER JOIN labs l ON l.id = s.lab_id
WHERE s.good < (
    SELECT AVG(s2.good) FROM stock s2 WHERE s2.item_id = s.item_id
)
ORDER BY s.good ASC LIMIT 4;
```

Table 7.5: Live Output from Correlated Subquery

stock_id	item_name	lab_name	available_units
4	DC Gear Motor	Robotics Lab	2
47	Rigol DS1054Z Digital Oscilloscope	AI Project Lab	2
24	Intel RealSense Depth Camera D435i	M.Tech Research Lab	3
70	Digital Multimeter MAS830L	M.Tech Research Lab	4

4. Indexing and Execution Plan Inspection (EXPLAIN FORMAT=TREE)

Execution plans generated via EXPLAIN FORMAT=TREE confirm high-performance index traversal on Aiven Cloud MySQL 8.4:

- **Primary Key Clustered Scan:** EXPLAIN FORMAT=TREE SELECT * FROM users WHERE id = 1;
  Result: -> Rows fetched before execution (cost=0..0 rows=1). Clustered B-Tree primary key lookup resolves in O(1) time.
- **Composite Secondary Index Scan:** EXPLAIN FORMAT=TREE SELECT * FROM stock WHERE item_id = 1 AND lab_id = 1;
  Result: -> Rows fetched before execution (cost=0..0 rows=1). Composite secondary unique index completely covers the predicate.

5. Transaction Strategy and ACID Integrity Demonstrations

Figure 7.1 illustrates the ACID transaction execution flow during hardware handover. It highlights the critical difference between database rollback (which guarantees ledger consistency and zero dirty reads) and physical custody operations:

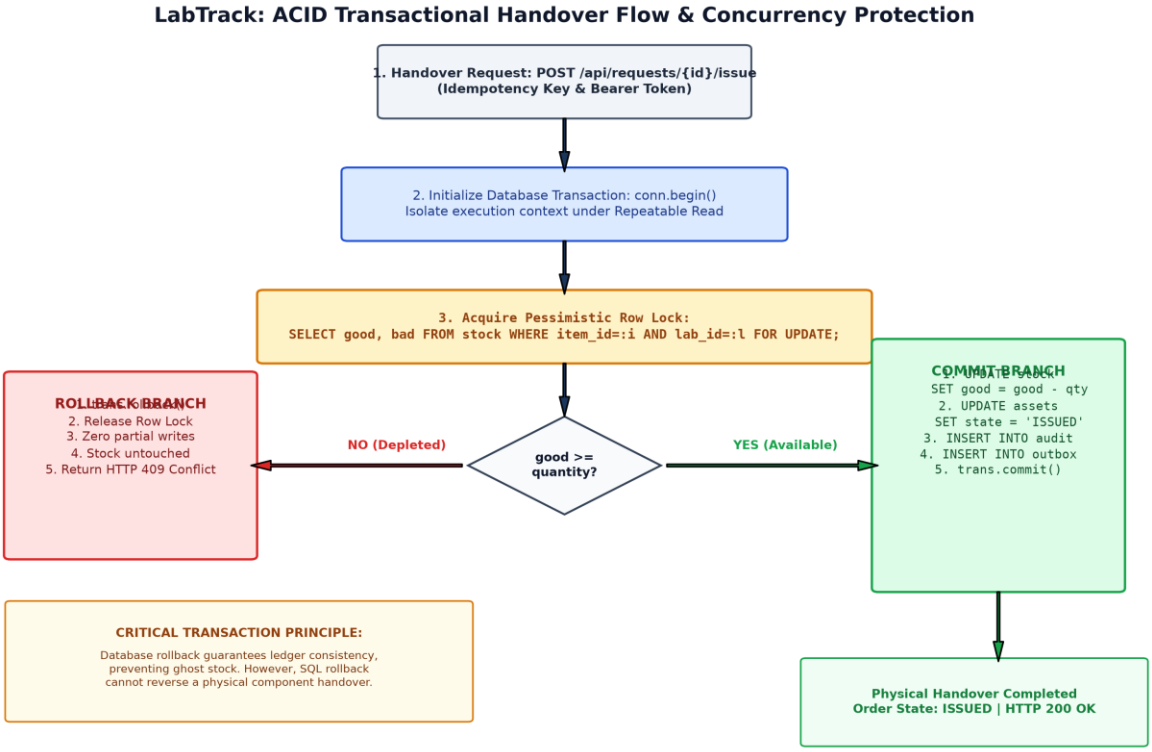


Figure 7.1: *LabTrack ACID Transactional Handover Flow, Row-Level Locking, and Rollback Protection*

Executed Concurrency Tests: Two comprehensive Python test scripts demonstrate ACID transactional guarantees under concurrent operational stress:

- **Demonstration 1 (Pessimistic Row Locking):** A transaction initiates with conn.begin(), executes SELECT good FROM stock WHERE item_id = 1 AND lab_id = 1 FOR UPDATE to acquire an exclusive pessimistic row lock, decrements good stock by 1, and issues trans.commit(). The balance was verified decremented from 7 to 6 atomically while blocking concurrent mutating transactions.
- **Demonstration 2 (Automatic Rollback):** A transaction initiates, decrements stock by 5 units, and encounters a simulated business logic exception (simulating unauthorized student borrowing). The exception triggers trans.rollback(). Live verification confirmed stock remained strictly at 7 units, proving zero dirty reads or partial writes.

6. MongoDB Atlas Event Projection and Aggregation Pipeline

Event Projection Architecture: Figure 7.2 illustrates the asynchronous data flow bridging the MySQL outbox table with MongoDB Atlas via the background worker, followed by the four-stage analytical aggregation pipeline:

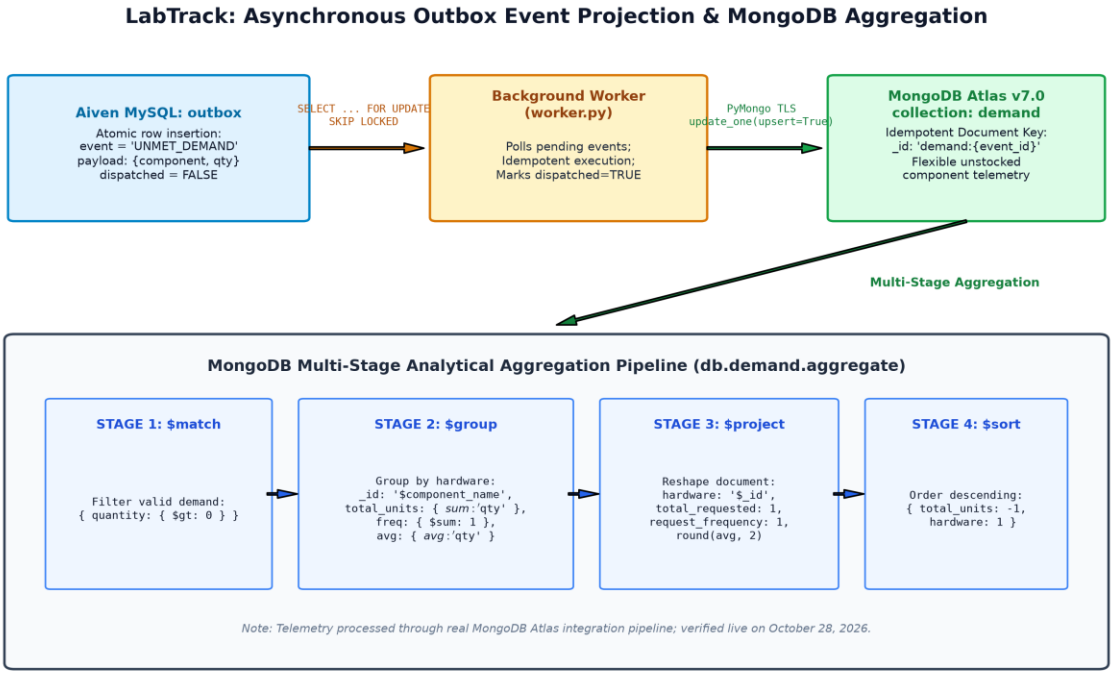


Figure 7.2: *LabTrack Asynchronous Outbox Event Projection and MongoDB Multi-Stage Aggregation Pipeline*

Listing 7.9: MongoDB Atlas Multi-Stage Aggregation Pipeline

```
db.demand.aggregate([
  { match: { quantity: { gt: 0 } } },
  { group: {
      _id: { ifNull: ["component_name", "name"] },
      total_requested_units: { sum: "quantity" },
      request_frequency: { sum: 1 },
      max_single_batch: { max: "quantity" },
      avg_units_per_req: { avg: "quantity" }
  }},
  { project: {
      _id: 0,
      hardware_component: "_id",
      total_requested_units: 1,
      request_frequency: 1,
      max_single_batch: 1,
      avg_units_per_req: { round: ["avg_units_per_req", 2] }
  }},
  { sort: { total_requested_units: -1 } }
]);
```

Table 7.6: Live Aggregated Student Demand from MongoDB Atlas Cluster (Synthetic Pipeline Verification)

Hardware Component Target	Total Units Requested	Request Frequency	Max Batch	Avg Units
NVIDIA Jetson Orin Nano 8GB	10	2	5	5.00
LiPo 3S 2200mAh 35C Battery	8	1	8	8.00
Ultrasonic Sensor HC-SR04	6	2	4	3.00
Waterproof Sensor JSN-SR04T	6	1	6	6.00

Hardware Component Target	Total Units Requested	Request Frequency	Max Batch	Avg Units
Intel RealSense D435i Depth Camera	3	1	3	3.00

7.4 Institutional Master Dataset and Sample Data

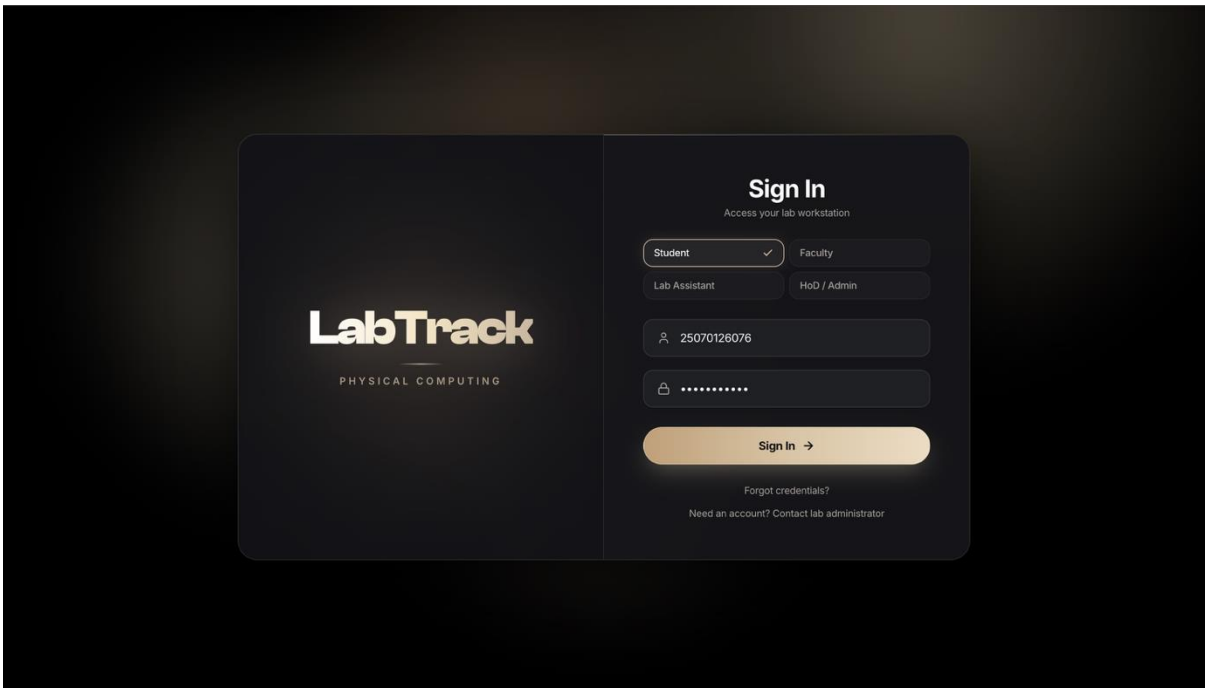
Institutional Dataset Scope: The relational database is populated with an authoritative institutional roster synchronized from labtrack_master_data.xlsx. The operational scale comprises:

- **Users:** 73 active institutional user accounts spanning Student, Mentor, In-Charge, Assistant, and HoD roles.
- **Laboratories:** 10 physical engineering laboratory spaces across SIT Pune departmental floors.
- **Catalogue Items:** 43 hardware component master types across 7 engineering categories.
- **Stock Balances:** 114 laboratory-specific stock balance allocations with segregated good/bad counts.
- **Physical Assets:** 107 individualized, serial-barcoded trackable hardware asset units.

CHAPTER 8
USER INTERFACE

8.1 Screenshots of the User Interface

The LabTrack frontend is an interactive single-page application built with React, TypeScript, and Vite. In accordance with institutional evaluation guidelines, authentic running interface captures are being compiled by the project team (Raghu). To preserve strict document typography, page layout, and numbering hierarchy, dedicated visual placeholders are established below, defining the target route, viewport specifications, and verification objective for each interface view:



Verification Objective: Verifies PRN/Staff identifier authentication and PBKDF2 password verification across institutional roles.

Figure 8.1: *LabTrack Institutional Authentication Portal*

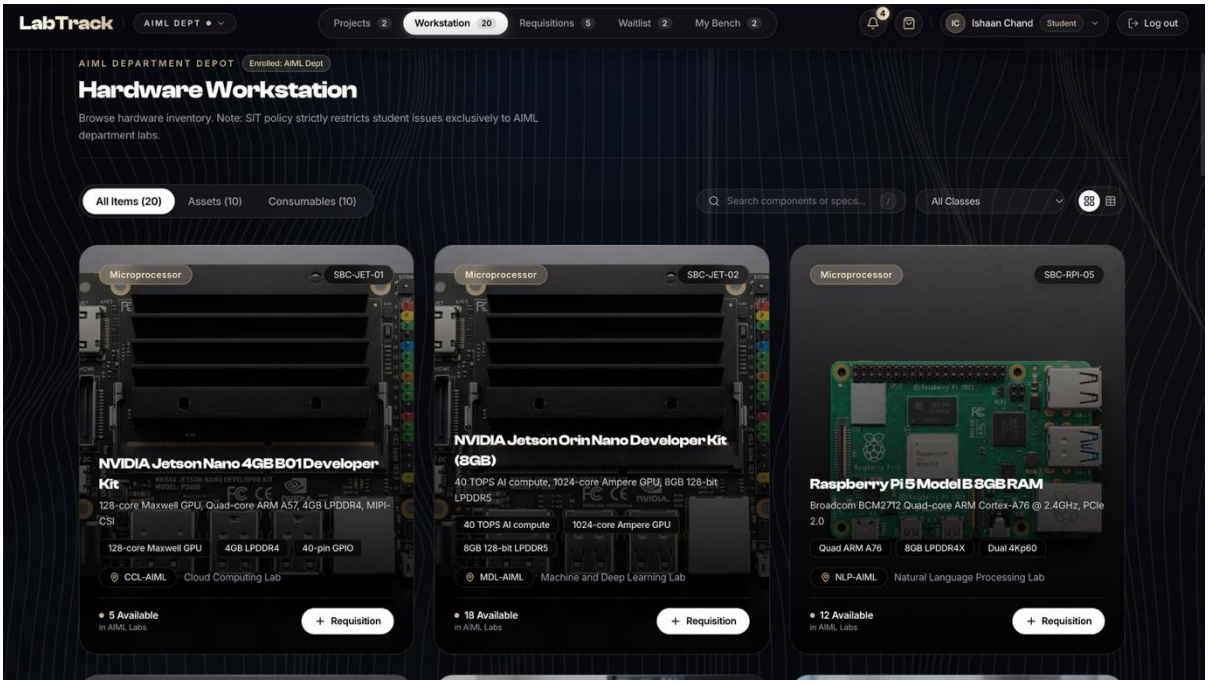


Figure 8.2: *Multi-Laboratory Hardware Catalogue and Search*

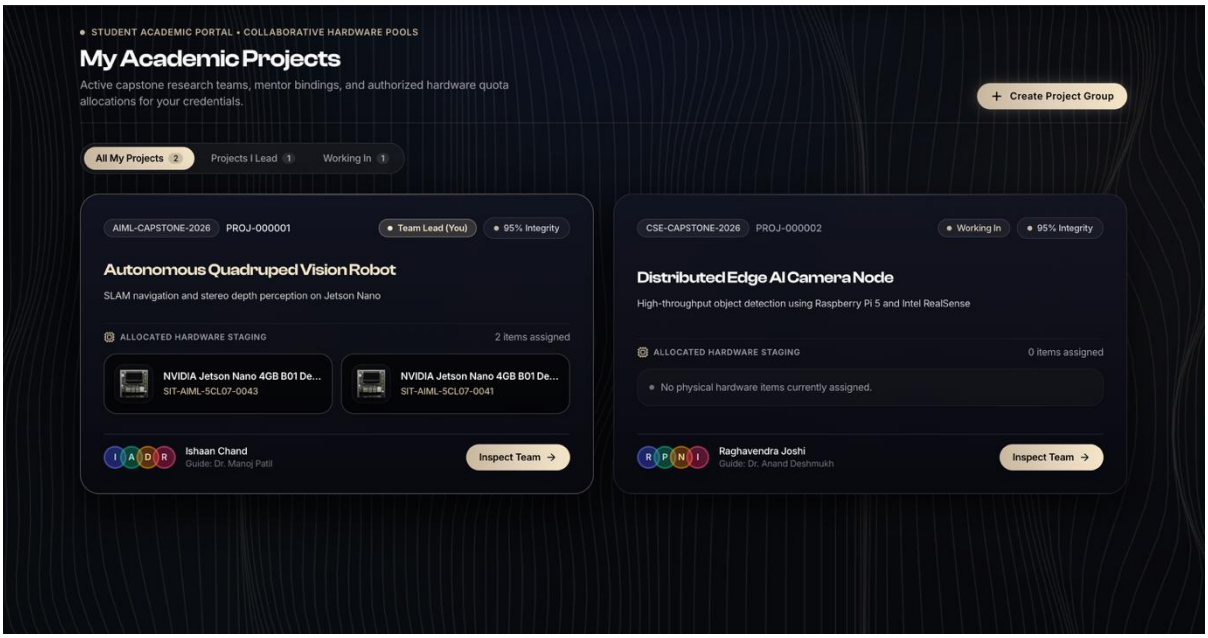


Figure 8.3: *Research Project Management Workspace*

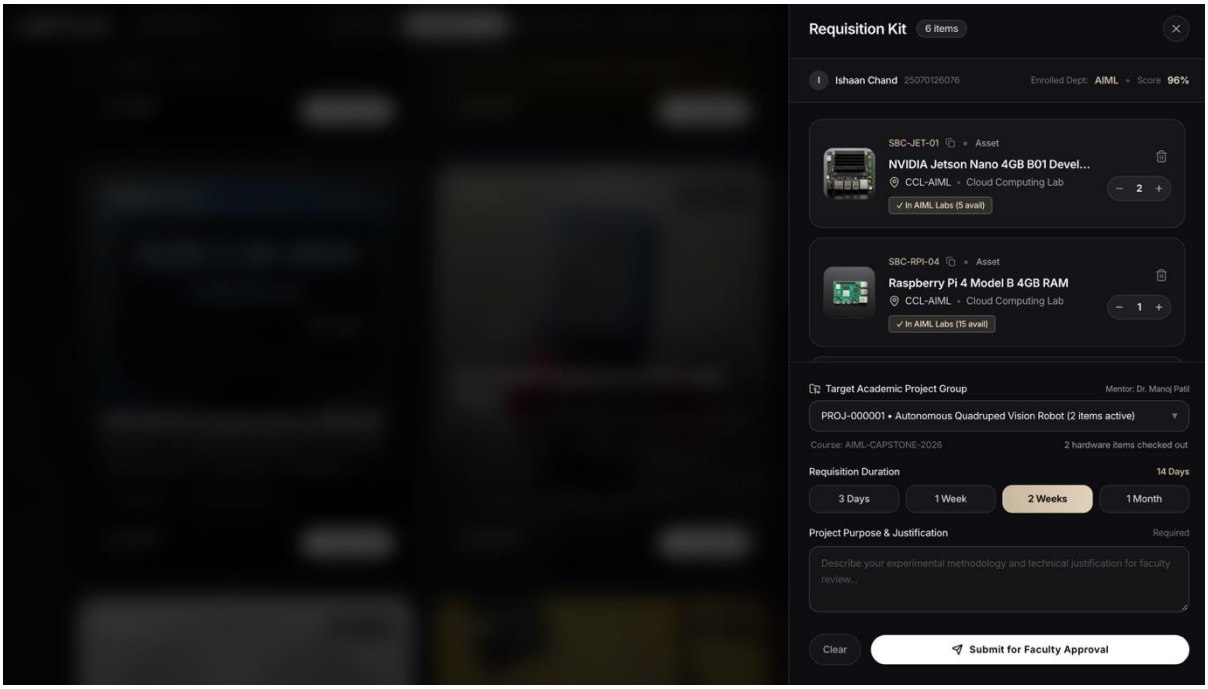


Figure 8.4: *Multi-Item Requisition Submission Interface*

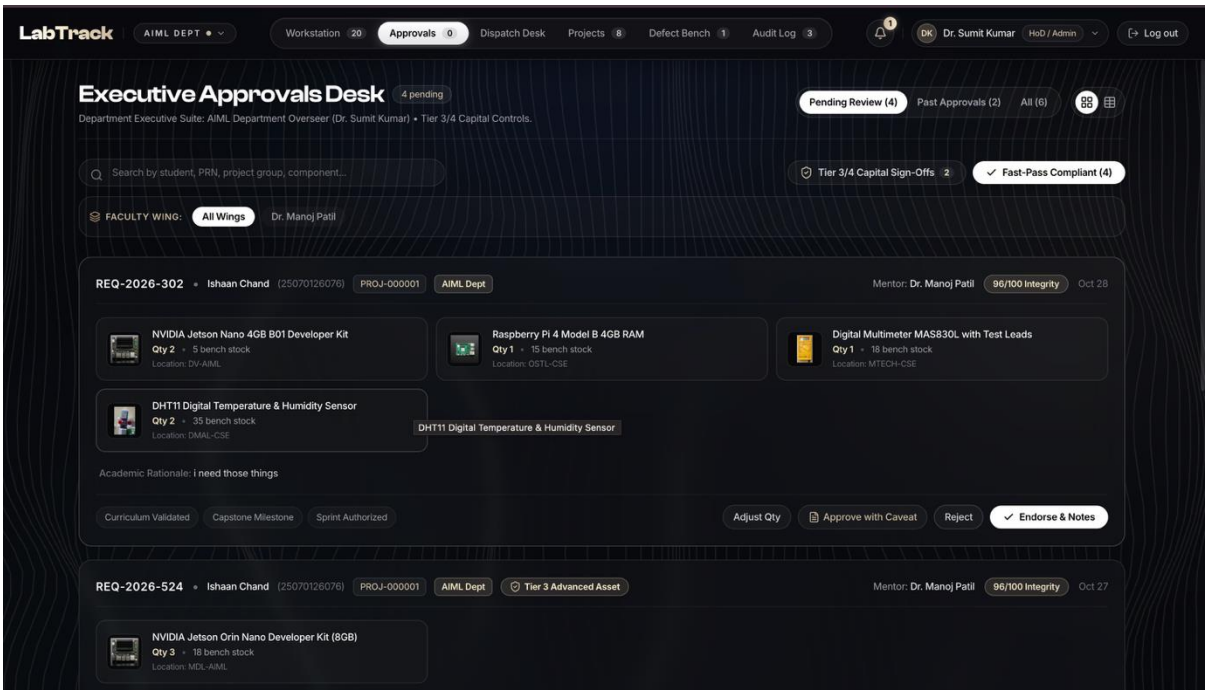


Figure 8.5: *Departmental Head (HoD) Tier-2 Multi-Tier Approval Review*

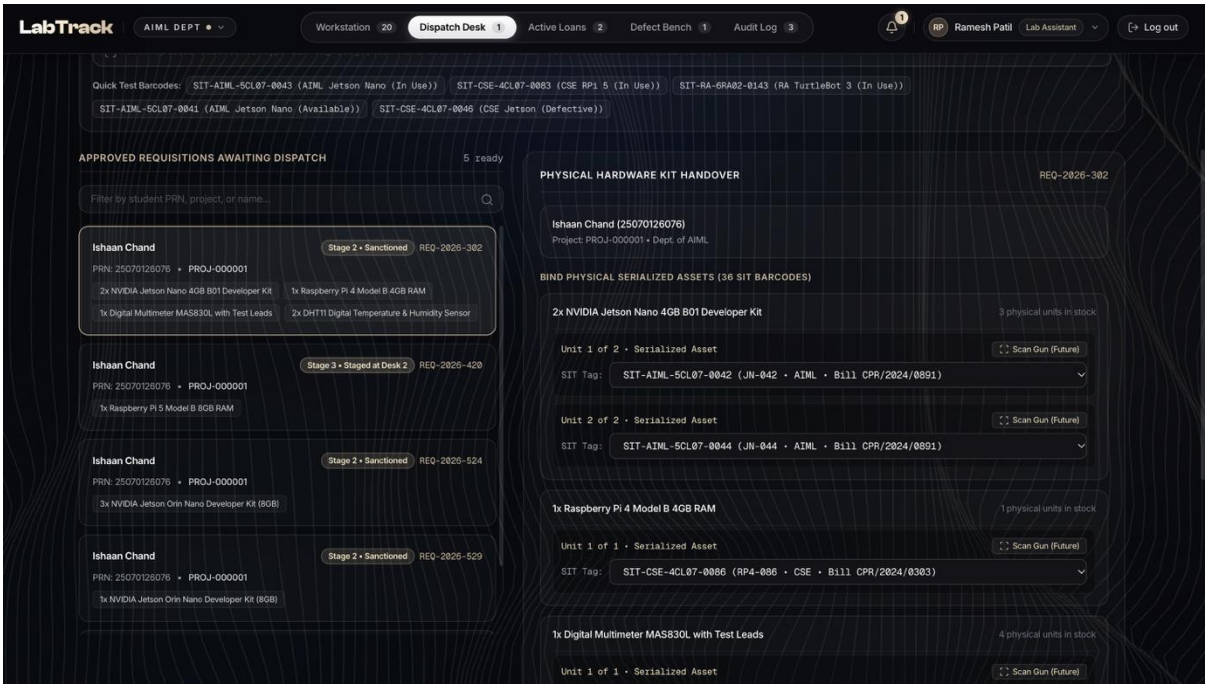


Figure 8.6: *Laboratory Service Counter Handover and Barcode Binding*

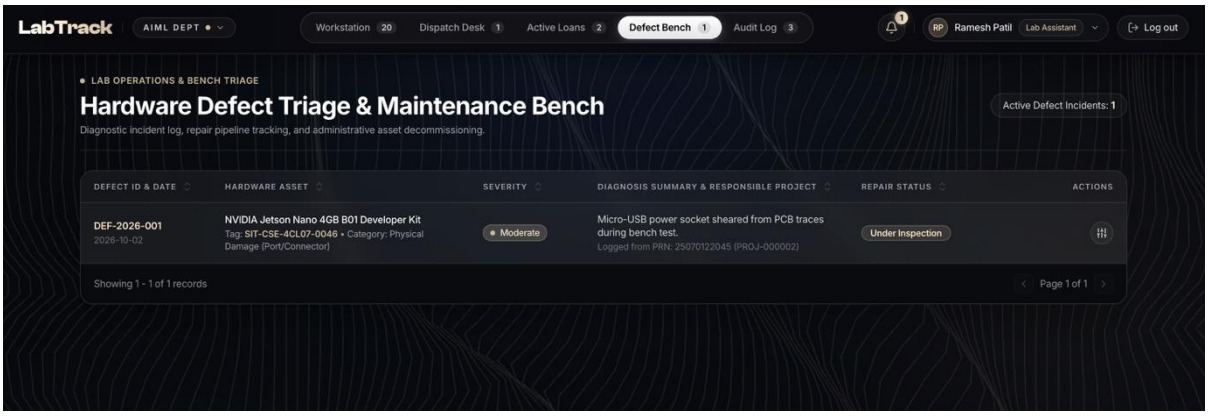


Figure 8.7: *Counter Return Technical Inspection and Condition Grading*

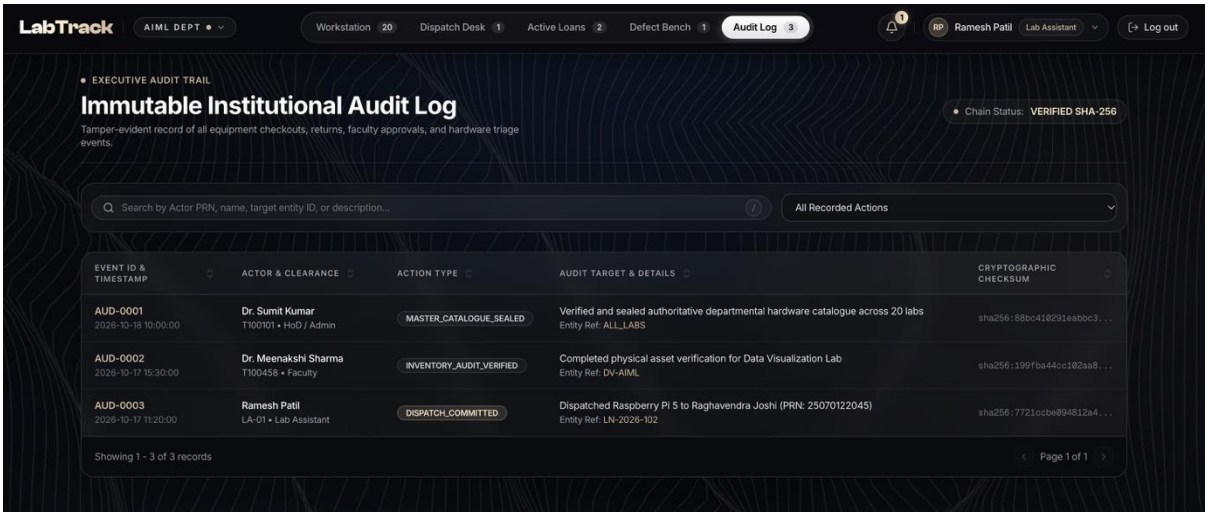


Figure 8.8: *Immutable Requisition Lifecycle Timeline and Audit Verification*

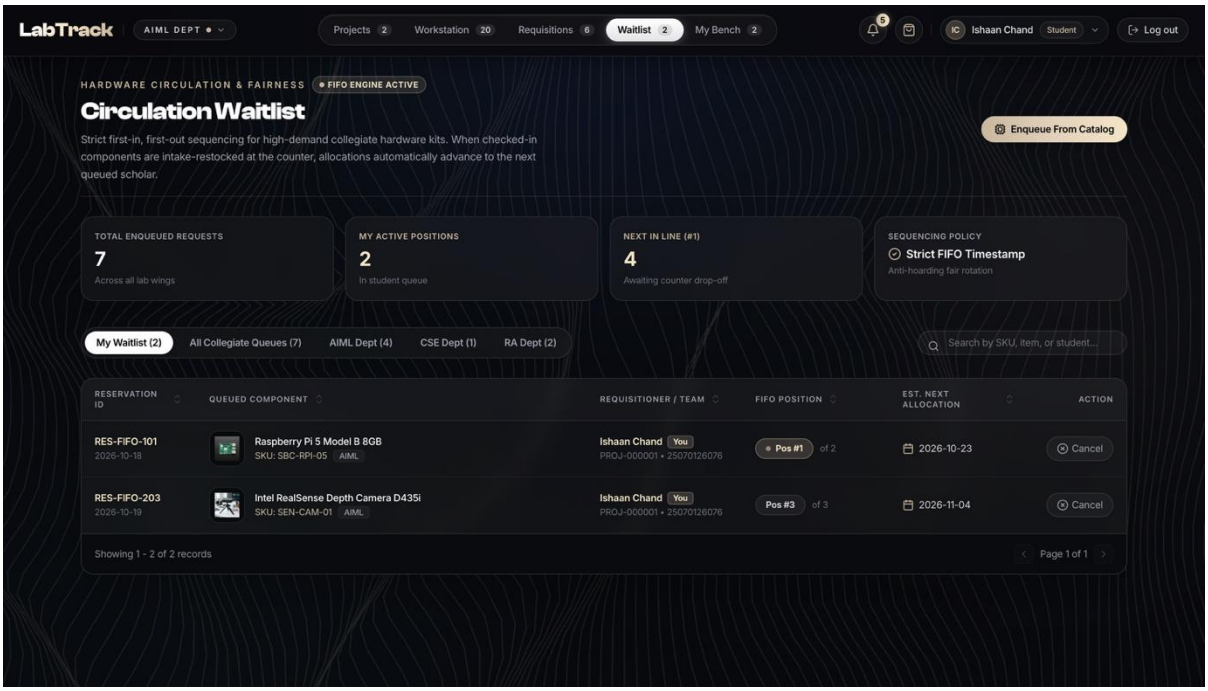


Figure 8.9: *Component Waiting Queue and Reservation Management*

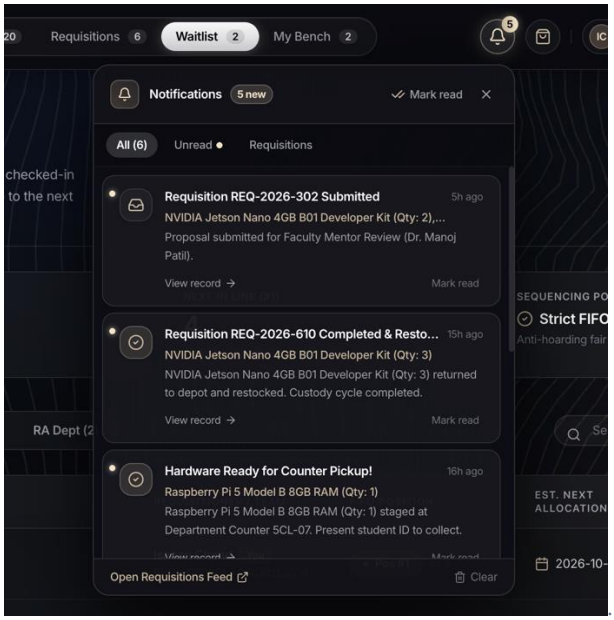


Figure 8.10: *Real-Time User Notification Feed Drawer*

8.2 Usage Instructions and Workflows

LabTrack provides streamlined, role-specific operational workflows guiding users through hardware lifecycle governance:

- **Student Workflow:** 1. Authenticate with student PRN. 2. Create or join an academic research project. 3. Browse catalogue and add components to requisition cart. 4. Submit order with justification and loan duration. 5. Present digital order at lab counter once approved for barcode handover.
- **Faculty Mentor Workflow:** 1. Access 'Requests' dashboard. 2. Filter pending student project proposals. 3. Review academic justification and hardware quantities. 4. Endorse request, advancing order to Laboratory In-Charge review.
- **Lab In-Charge Workflow:** 1. Review laboratory stock availability. 2. Validate component suitability for project scope. 3. Grant lab allocation approval. 4. Review incident escalations and damaged component reports.
- **Lab Assistant Workflow:** 1. Open counter queue. 2. Claim approved requisition order. 3. Physically pick components and scan asset barcodes. 4. Confirm digital issuance. 5. Receive returned components, execute functional inspection (PASS/FAIL), and confirm return reconciliation.
- **Head of Department Workflow:** 1. Access executive governance portal. 2. Review Tier-2 escalated requisitions (computing boards, depth sensors). 3. Evaluate departmental budget and resource utilization. 4. Grant Tier-2 authorization. 5. Access unmet demand analytics.

Automated Verification and Quality Assurance

Test Suite Summary: The platform's reliability is validated through an extensive automated test suite comprising 77 test cases across five test modules:

- **Core API Tests:** tests/test_api.py: 64 passed. Verifies all REST endpoints, auth tokens, RBAC permissions, and requisition lifecycles.
- **Migration Tests:** tests/test_migration.py: 4 passed. Validates SQLite to MySQL 8.4 schema synchronization, foreign key integrity, and row reconciliation.
- **Authentication Smoke Tests:** scripts/smoke-test-excel-auth.cjs: 11 passed. Validates authentic PBKDF2 authentication across all 73 institutional roster accounts.
- **Outbox Projection Tests:** scripts/verify_mongo_demand_projection.py: 1 passed. Validates outbox event polling, dispatching, and MongoDB aggregation.

CHAPTER 9
CONCLUSION

9.1 Summary of Technical Achievements

The LabTrack platform successfully demonstrates the design, implementation, and deployment of a full-scale institutional hardware lifecycle, requisition, and custody governance platform tailored to the Department of Artificial Intelligence & Machine Learning at Symbiosis Institute of Technology. By replacing fragmented paper ledgers and isolated spreadsheets with an integrated, polyglot persistence architecture, LabTrack resolves longstanding institutional bottlenecks.

Key Outcomes: Key technical milestones achieved include:

- **Inventory Visibility:** Complete elimination of phantom stock through real-time multi-laboratory inventory synchronization.
- **Approval Velocity:** Streamlined multi-tier academic approval workflows, reducing requisition turnaround latency from days to minutes.
- **Custody Accountability:** Definitive barcode-level custody traceability, ensuring accountability for fragile and high-value edge computing platforms.
- **Database Robustness:** Strict transactional integrity guaranteed through MySQL 8.4 ACID compliance and pessimistic row-level locking.
- **Procurement Intelligence:** Data-driven hardware procurement signals unlocked via asynchronous Transactional Outbox projections and MongoDB Atlas aggregation.

9.2 Database Engineering Lessons Learned

Pedagogical Insights: Development of LabTrack provided deep practical experience across foundational database engineering disciplines:

- **Relational Modeling:** Conceptual ER modeling and translation into Boyce-Codd Normal Form (BCNF) relational structures.
- **Polyglot Architecture:** Strategic justification of polyglot architectures, leveraging relational engines for ACID ledgers and document stores for polymorphic datasheets.
- **Concurrency Management:** Concurrency control through pessimistic row-level locking (SELECT ... FOR UPDATE) and automatic transaction rollbacks.
- **Performance Tuning:** Query optimization through execution plan analysis (EXPLAIN FORMAT=TREE) and composite index tuning.

9.3 Prospective Future Scope

Future Roadmap: Planned enhancements for prospective subsequent developmental phases include:

- **IoT Smart Lockers:** RFID and IoT Smart Locker integration to enable automated 24/7 self-service component pickup and drop-off without counter attendants.
- **Predictive Demand ML:** Machine Learning predictive demand forecasting analyzing academic syllabi, course timetables, and capstone proposal trends.
- **Mobile Scanning App:** Native mobile application with integrated optical camera QR/barcode scanning for laboratory assistants on floor duty.

- **Prospective Institutional Pilot:** Pilot deployment within SIT R&A laboratories, subject to formal institutional approvals and laboratory committee review.

CHAPTER 10
REFERENCES

10.1 Academic and Technical References

The implementation and architectural design of LabTrack are grounded in established academic literature and industry specifications:

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed., New York, NY, USA: McGraw-Hill Education, 2020.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Boston, MA, USA: Pearson, 2017.

[3] M. Fowler, Patterns of Enterprise Application Architecture, Boston, MA, USA: Addison-Wesley Professional, 2002.

[4] Oracle Corporation, MySQL 8.4 Reference Manual, Redwood City, CA, USA: Oracle Corporation, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/

[5] MongoDB Inc., MongoDB Atlas Database and Aggregation Framework Documentation, New York, NY, USA: MongoDB Inc., 2026. [Online]. Available: https://www.mongodb.com/docs/manual/

[6] SQLAlchemy Authors, SQLAlchemy 2.0 Unified Documentation and Expression Language, 2026. [Online]. Available: https://docs.sqlalchemy.org/en/20/

[7] S. Ramirez, FastAPI: High Performance Asynchronous Web Framework for Python, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[8] Symbiosis Institute of Technology, Database Concepts for Data Science Lab (DCDS) Miniproject Rubric and Laboratory Guidelines, Department of Artificial Intelligence & Machine Learning, SIT Pune, 2025 - 2026.

[9] I. R. Chand, LabTrack: Institutional Hardware Lifecycle, Requisition & Custody Management System, GitHub Repository, 2026. [Online]. Available: https://github.com/irxchand/LabTrack